DATE: 17/07/2026

EX NO: 01

LINUX COMMANDS AND WINDOWS COMMANDS

AIM:

To study and execute various linux commands for file and directory management 1. text processing, and basic networking operations.

Study of varias commands used in linux

pwd

Description: prints the current working directory

Output:

$ pwd

/home/Liveuser

mkdir

Description: creater a new directory

Output:

$ mkdir fedora

$ ls

Desktop : Documents Downloads fedora

cd

Description: changes the current directory.

Output:

$ cd fedora

$ pwd

/home/liveuser/fedora

touch filename.txt

Description: Creates an empty file.

Output:

```
$ touch example.txt
$ ls
example.txt
```

echo "Hi" > filename.txt

Description: Write "Hi" to given file, overwriting any existing content using echo.

Output:

```
$ echo "Hi" > example.txt
cat filename.txt
```

Description: Displays the contents of a file.

Output

```
$ cat example.txt
Hi
```

grep

Description: Searches for a specific pattern in a file.

Output:

```
$ grep "H" example.txt
Hi
```

wc

Description: Displays the number of lines, words, and bytes in a file.

Output:

```
$ wc example.txt
1 1 3 example.txt
```

wc -l

Description: counts the number of lines in a file.

Output:

```
$ wc -l example.txt
1 example.txt
```

wc -w

Description: counts the number of words in a file.

Output:

```
$ wc -w example.txt
example.txt
```

wc -c

Description: counts the number of bytes in a file.

Output:

```
$ wc -c example.txt
3 example.txt
```

wc -m

Description: Counts the number of characters in a file.

Output:

```
$ wc -m example.txt
3 example.txt
```

wc -L

Description: Displays the length of the longest line in a file.

Output:

```
$ wc -L example.txt
2 example.txt
```

BASIC NETWORKING COMMANDS

hostname

Description: Displays the system's hostname.

Output:

```
$ hostname
localhost - live
route
```

route

Description: Displays the routing table.

Output:

```
$ route
Kernel IP routing table
```

netstat

Description: shows network connections and routing information.

Output:

```
$ netstat
Active internet connections
```

ip address show

Description: Displays IP address and network interface details.

Output:

```
$ ip address show
1 : lo:
2 : enp0s3:
```

nslookup

Description: Finds the IP address of a domain.

Output:

```
$ nslookup www.google.com
Name : www.google.com
Address : 142.250.xxx.xxx
```

ifconfig

Description: Displays network interface configuration.

Output:

```
$ ifconfig
enp0s3
inet 10.0.2.15
```

ping

Description: Tests connectivity to a host.

Output:

$ ping www.google.com
64 bytes from ...

arp

Description: Displays the ARP cache.

Output:

$ arp -a
-gateway (10.0.2.2)

traceroute

Description: Displays the route taken by packets to reach a destination.

Output:

$ tracerroute www.google.com
1. - gateway
2 ...

RESULT:

Thus the Linux and windows commands are studied and executed successfully.

17/7/26

DATE: 23/07/2026

Exp No 2

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTER

AIM:

To know IP addressing and manually assign an IP address to a computer. To verify the assigned IP address using Linux networking commands.

THEORY:

An IP addres is a unique numerical addres assigned to every device connected to a network. It enables communication between device over a network. An IPv4 address consists of four octets seperated by dots (eg: 192.168.1.10)

An IP addres can be assigned either manually (static IP) or automatically (Dynamic IP) using DHCP. A manually assigned IP address remains unless changed by the user along with the IP address, parameters such as the Subnet Mask, Default Gateway, and DNS server are also configured for proper network communication.

In Linux, the ip command is used to display and manage IP addres, network interfaces, routing tables, and neighbouring devices. It is one of the most commonly used commands for network configuration and troubleshooting.

Commands and Sample Outputs

1. Display all IP address

command:

ip addr show

sample output

2: ens160:

inet 172.16.211.128/24

inet6 fe80::20c:291f::fe75:9927/64

observation:

Displays the IPv4 and IPv6 addresses of all networks.

2. Display a specific network interface:

Command:

ip addr show dev ens160

Sample output:

2: ens160:

inet 172.16.211.128/24

3. Display Routing Table

command:

ip route show

sample output:

default via 172.16.211.2 dev ens160

172.16.211.0/24 dev ens160

observation:

Displays the routing table and the default gateway.

4. Display Neighbar (ARP) Table

command:

ip neigh show

Sample output

172.16.211.1 REACHABLE

172.16.211.254 STABLE

observation:

Displays the neighbouring devices and this MAC address

RESULT

The IP address user manually assigned and verified successfully using Linux networking commands.

DATE: 23/07/2026

EXP NO: 3

STUDY OF DIFFERENT TYPES OF NETWORK CABLES AND CRIMPING OF CABLE WITH RJ45 Connectors

AIM:

To study different types of network cables and understand the crimping process using an RJ45 connector.

CONCEPT:

Network cables are used to connect computers and other networking devices for data communication. The most commonly used cable in LAN is the unshielded Twisted pair (UTP) cable. An RJ45 connector is attached to the ends of the cable to establish a network connection. The wires are arranged according to the T568A or T568B wiring standard before crimping, and the completed cable is tested to ensure proper connectivity.

Types of Network cables:

Cable Type	Description
1. UTP (unshielded Twisted pair)	commonly used for Ethernet LAN's and office networks
2. STP (shielded Twisted pair)	used in area with high electrical or electromagnetic interference
3 Coaxial cable	used for cable television, broadband internet, and CCTV systems.
4. Optical fiber cable	Used for high speed, high bandwidth and long distance communication.

CRIMPING PROCEDURE

1. Cut the UTP cable to the required length.
2. Strip about 2-3 cm of the outer insulation.
3. Untwist and straighten the wires.
4. Arrange the wires according to the T568B standard.
5. Trim the wire ends evenly.
6. Insert the wires into the RJ45 connector.
7. Crimp the connector using a crimping tool.
8. Test the cable using a LAN cable tester.

Observation

* The wires were arranged according to the T568B standard.
* The RJ45 connector was crimped successfully.

* The cable tester indicated proper continuity.

* The cable was ready for network communication

RESULT:

Two different types of network cables were studied, and the UTP cable was successfully terminated using and RJ45 connector and tested for proper connectivity.

DATE: 31/08/26

Ex No: 4

IMPLEMENT PACKET SNIFFING USING RAW SOCKETS IN PYTHON

AIM:

To implement packet sniffing using raw sockets in python and capture network packets passing through the network interface

OBJECTIVE

To understand how raw sockets work and how network packets can be captured and analyze using python programming.

THEORY:

1. Packet Sniffing is the process of monitoring and capturing data packets travelling across a network.

2. A packet sniffer intercepts packets and analyzes their contents.

2. Raw sockets provide direct access to lower-level network protocols, allowing applications to capture packets before they are processed by the operating system.

4. In python, the socket module, can be used to create raw sockets for packet capture.

5. This technique is useful for network monitoring, troubleshooting, and security analysis.

ALGORITHM:

1. Import the required socket module.
2. Create a raw socket.
3. Bind the socket to the network interface.
4. Receive packets using the recvfrom() method.
5. Display the captured packet information.
6. Repeat the process continuously to monitor network traffic.

PROGRAM:

```
import socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW,
          socket.IPPROTO_IP)
host = socket.gethostbyname(socket.gethostname())
sniffer.bind((host, 0))
sniffer.setsocket(socket.IPPROTO_IP, socket.IP_HDRINCL
while True:
    raw_data, addr = sniffer.recvfrom(65535)
        print("Packet received from:", addr)
        print(raw_data)
```

OUTPUT:

Packet received from:
('192.168.1.5', 0),
b'E\x00\x00 (...'

Packet received from.
('192.168.1.10', 0)
b'E\x00\x00(...'

RESULT:

Thus, packet sniffing was successfully implemented using raw sockets in python, and network packets were captured and displayed.

DATE: 24/07/2026
EXP NO: 05

DEVELOP A CUSTOMIZED PING COMMAND TO TEST SERVER CONNECTIVITY

AIM:

To develop and execute a customized ping command for testing server connectivity and analyzing network performance.

THEORY:

The ping command is a network diagnostic tool used to determine whether a remote server or host is reachable. It works by sending ICMP (Internet Control Message Protocol) Echo Request packets to the destination and waiting for Echo Reply packets. The command displays the response time, packet loss, and network statistics, helping users troubleshoot connectivity problems.

Commands

1. ping google.com

Output

Pinging google.com [142.250.77.110] with 32 bytes of data:
Reply from 142.250.77.110: bytes=32 time=6ms TTL=119
Reply from 142.250.77.110: bytes=32 time=16ms TTL=119
Reply from 142.250.77.110: bytes=32 time=5ms TTL=119

Ping statistics for 142.250.77.110
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 5ms, Maximum = 16ms, Average = 8ms

2. ping localhost

Output:
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

3. ping 8.8.8.8

Output:
Reply from 8.8.8.8: bytes=32, time=25ms TTL=118
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

4. ping -n 5 google.com

Output:
Packets: Sent = 5, Received = 5, Lost = 0 (0% loss)
Average = 20ms

5. ping 192.168.1.1

Output:
Reply from 192.168.1.1: bytes=32 time<1ms TTL=64
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

1. Why is the Ping command used

The Ping command is used to check whether a remote host or server is reachable over a network and to diagnose network connectivity issues.

2. What is the purpose of the Ping command

The purpose of the Ping command is to

i) Verify network connectivity

ii) Measure response time

iii) Detect packet loss.

3. How is the Ping command established?

The ping command works by

i). sending an ICMP Echo Request packet to the destination host.

ii) The destination host receives the request

iii) If reachable, it returns an ICMP Echo Reply packet.

iv) The sender measures the round-trip time and displays network statistics.

RESULT:

The customized Ping command was successfully executed. The server was reachable, with 0% packet loss, and reliable network connectivity

DATE: 20/08/2026

Ex No: 9

DESIGN A SIMPLE TOPOLOGY AND CONFIGURE WITH ONE ROUTER, TWO SWITCHES AND PCS USING CISCO PACKET TRACER

AIM:

To design and configure a simple network topology consisting of one router, two switches, and multiple PCs in Cisco Packet Tracer, and enable communication between PCs.

DEFINITIONS:

1. Router: A networking device that forwards data packets between computer networks, directing traffic on the Internet.

2. Switch: A device that connects devices within a network and uses MAC addresses to forward data to the correct destination.

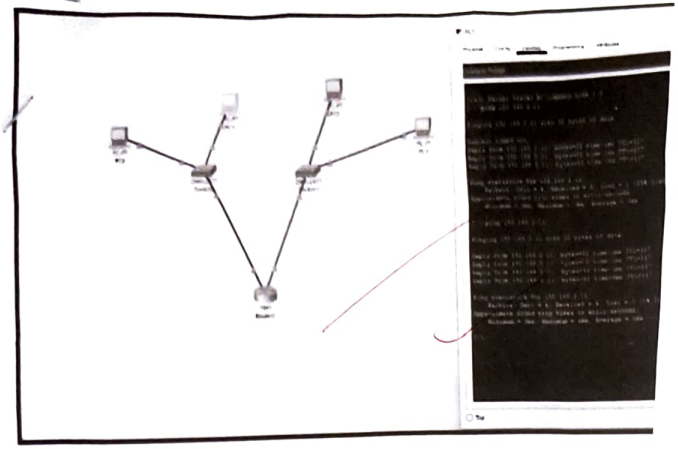
3. PC (Personal Computer): End devices in a network that users interact with to send and receive data.

4. Cables:

i) Straight-through cable

ii) crossover cable

iii) Console cable



PROCEDURE:

1. Open Cisco Packet Tracer
2. Add Devices to Workspace
3. Connect Devices with Cables
4. Configure Router Interfaces
5. Configure PC IP Addresses
6. Save and Test Connectivity

RESULT:

The PCs connected to different switches can communicate with each other via a router. The successful ping replies verify the network is properly configured

DATE: 21/08/2026

EX NO: 10

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

AIM:

To learn how to add a network module to a Cisco switch, connect multiple PCs and a server, assign IP addresses, and test connectivity in Cisco Packet Tracer.

DESCRIPTION:

This experiment is about building a simple computer network using a switch with extra ~~points~~ module ports. You will connect five PCs and one server, assign IPs and check if all devices can communicate. This type helps you understand how module switches are used to create flexible networks.

DEFINITION

A network module is an extra component you can add to a switch. It gives you more ports or adds new types of connections. In Packet Tracer, modules such as HWIC-4ESW let you expand the switch's capabilities.

PROCEDURE

1. Open Cisco Packet Tracer and set up workspace

2. Drag and drop a 2960-24TT switch to the center

3. Add five PC's and one server PT from end devices panel

4. Power off the switch.
   * click the switch, open physical tab, and turn off the power button

5. Install HWIC-4ESW module on the switch.
   * In the physical tab, drag HWIC-4ESW into an empty module slot

6. Power on the switch and let it boot

7. Connect all the devices using copper straight through cables.

   PC_0 - Switch Fast Ethernet 0/1
   PC_1 - Switch Fast Ethernet 0/2
   PC_2 - switch Module Port 1
   PC_3 - switch Module Port 2
   PC_4 - switch Module Port 3

   Server 0 - switch module port 4 &

   Assign IP addresses to each device:

   PC_0 : 192.168.1.10
   PC_1 : 192.168.1.11
   PC_2 : 192.168.1.12
   PC_3 : 192.168.1.13
   PC_4 : 192.168.1.14
   Server 0 : 192.168.1.100
   All subnet masks : 255.255.255.0

8. Test connectivity (From any PC - open command prompt and ping other devices)

Input:

Devices :

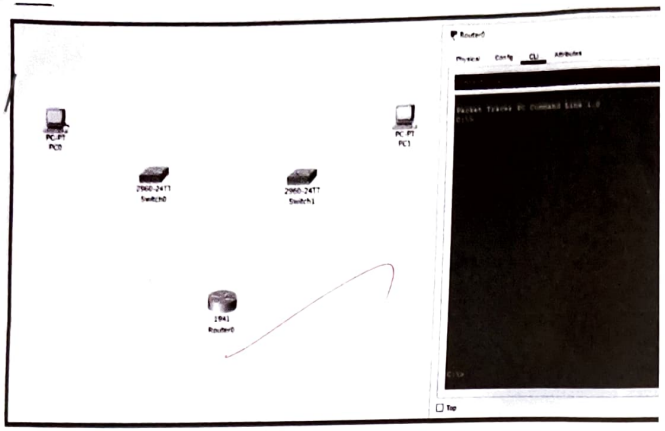
1 Cisw 2960 - 24TT switch
1 HWIC-4ESW network module
5 PC_s (PC_0 - PC_4)
1 server - PT (Server 0)

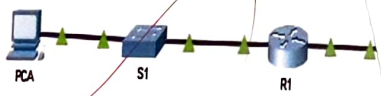
cables :

1. copper straight through cables IP configuration.
2. Use static IP addresses as listed above.

Output:

A working network setup where every PC and then the server can successfully ping each other. Proving that both built-in and module ports on the switch are functioning.





RESULT:

The experiment demonstrates successfully customization of a switch with a network module and connectivity between all devices. The networks works an expected and all devices communicate.

DATE: 27/08/2026
EX NO: 7

DEVELOP A SIMPLE CALCULATOR USING XML-RPC

AIM:

To develop a simple calculator application using XML-RPC (Remote Procedure Call) that performs basic arithmetic operations such as addition, subtraction, multiplication, and division between a client and a server.

SOFTWARE REQUIREMENTS

1. Python 3.x
2. XML-RPC Library
2. Any Python IDE or Command Prompt

THEORY

XML-RPC is a protocol that allows a client application to call methods on a remote server over a network. The request and response data are encoded using XML and transmitted using HTTP.

In this experiment, the XML-RPC server provides four methods which are addition, subtraction, multiplication, and division. The client sends the required values and operation to the server, and the server performs the calculation and returns the result.

ALGORITHM:

Server

1. Import the XML-RPC server module
2. Create an XML-RPC server.
3. Define functions for addition, subtraction, multiplication and division.
4. Register the functions with the server.
5. Start the server and wait for client requests.

Client:

1. Import the XML-RPC client module.
2. Connect to the XML-RPC server.
3. Accept two numbers and an operation from the user.
4. Call the corresponding remote method.
5. Display the result returned by the server.

CODE:

Server-side code (server.py)

```
from xmlrpc.server import SimpleXMLRPCServer
class Calculator:
    def add(self, x, y):
        return x + y
    def subtract(self, x, y):
        return x - y
    def multiply(self, x, y):
        return x * y
    def divide(self, x, y):
        if y != 0:
            return x / y
        else:
            return "Error: Division by zero is not allowed"

server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC server is running on port 8000...")
server.register_instance(Calculator())
server.serve_forever()
```

Client-Side code (client.py)

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
print("Addition: 10 + 5 =", proxy.add(10, 5))
print("Subtraction: 10 - 5 =", proxy.subtract(10, 5))
print("Multiplication: 10 * 5 =", proxy.multiply(10, 5))
print("Division: 10 / 5 =", proxy.divide(10, 5))
print("Division by zero: 10 / 0 =", proxy.divide(10, 0))
```

PROCEDURE:

1. Open a Python IDE or terminal
2. Create two seperate Python files: server.py and client.py
3. Write the provided server-side code in server.py and client-side code in client.py
4. Run the server.py file first to start the XML-RPC server.
5. Run the client.py file to perform arithmetic Operations remotely

6. Observe the results displayed on the client screen and verify the server output.

main.py

```
import threading
import time
import xmlrpc.client
from xmlrpc.server import SimpleXMLRPCServer
def run_server():
    server = SimpleXMLRPCServer(("localhost", 8000),
    server.register_function(lambda x, y: x + y, "ad
    server.register_function(lambda x, y: x - y, "sub
    server.register_function(lambda x, y: x * y, "mul
    server.register_function(lambda x, y: x / y if y
    Division by zero", "divide")
    server.serve_forever()
server_thread = threading.Thread(target=run_server, da
server_thread.start()
time.sleep(1)
proxy = xmlrpc.client.ServerProxy("http://localhost:80
print("--- XML-RPC Calculator Test ---")
num1, num2 = 30, 10
```

Output

```
--- XML-RPC Calculator Test ---
Inputs: 30 and 10
Addition (+)       : 40
Subtraction (-)    : 20
Multiplication (*): 300
Division (/)       : 3.0
```

RESULT:

A simple calculator application using XML-RPC that performs basic arithmetic operations are executed and verified successfully.

DATE: 31/7/25
Ex No: 6

BUILDING ANONYMOUS FTP SCANNER USING FTPLIB MODULE

AIM:

To create a Python-based FTP scanner using the ftplib module that checks if a public FTP server allows anonymous login. Lists the contents of directories on the server.

PROCEDURE

1. Import the built-in ftplib module.
2. Write two Python scripts:
   i) Sender side: Connects to the FTP server, logs in anonymously, and lists the root directory.
   ii) Receiver side: Connects anonymously and lists a specific sub-directory.
3. Run the scripts in Google Colab or any Python environment.
4. Observe the login status, server message, and directory contents.

CODE

SENDER SIDE CODE

```
#/usr/bin/env python3 import
ftplib
```

```
def anonymous_login(hostname):
    try:
        ftp = ftplib.FTP(hostname)
        = ftp.login('anonymous', 'anonymous')
        print(response)
        if "230" in response: # Login success code
            print("[+] " + str(hostname) + " FTP Anonymous
            Login Succeeded.")
            print("Welcome message: \n", ftp.getwelcome())
            print("\ Directory Listing: ")
            ftp.dir()
            ftp.quit()
        else:
            print("[-] " + str(hostname) + " FTP Anonymous
            Login Failed.")
            print("Error: ", str(e))
hostname = 'ftp.be.debian.org'
anonymous_login(hostname)
```

Receiver Side Code:

```
import ftplib
def check_anonymous_connection(host, path):
    try:
        with ftplib.FTP(host) as connection:
            connection.login('anonymous', 'anonymous')
            print("Welcome:", connection.getwelcome())
            print(f"\n listing directory: {path}")
            connection.cwd(path) for name, details in
            connection.mlsd():
                print(name, details['type'], details.get('size',
                'N/A'))
    except Exception as e:
        print("Error:", str(e))
```

```
FTP_SERVER_URL = 'ftp.be.debian.org'
'/pub/linux/kernel/v5.x/'
check_anonymous_connection(FTP_SERVER_URL, DOWNLOAD_DIR_PATH)
```

OUTPUT (SAMPLE)

SENDER SIDE OUTPUT:

230 Anonymous access granted, restrictions apply [+]
ftp.be.debian.org FTP Anonymous Login Succeeded.
Welcome Message:
220 ProFTPD Server (mirror.as35701.net) Directory
Listing:
drwxr-xr-x 9 ftp ftp 4096 Aug [redacted] 8:30 deb
....

RECEIVER SIDE OUTPUT:

Welcome: 220 ProFTPD Server (mirror.as35701.net)
Listing directory: /pub/linux/kernel/v5.x/ linux-5.10.1.tar
.xz file 113072564 linux-5.10.2.tar.xz file
[redacted] 113167444

RESULT:

The script successfully connected to the FTP server and verified anonymous login is allowed. The directory contents were retrieved and displayed. Hence, it proves that the server provides public access and is working correctly.

DATE: 25/09/2026
Ex No: 12

Nmap to discover Live Hosts using ARP scan, ICMP scan, and TCP/UDP Ping scan in TryHackMe Platform

Aim:

To discover live/active hosts in a network using Nmap by performing ARP scan, ICMP scan and TCP/UDP ping scan in TryHackMe platform.

ARP - Address Resolution Protocol

ARP is used to find the MAC address of a device from its IP address within a local network.

ICMP - Internet Control Message Protocol

ICMP is a network protocol used for error reporting and network protocol diagnostics. The ping command commonly uses ICMP Echo Request and Echo Reply messages.

TCP - Transmission Control Protocol

TCP is a connection-oriented transport protocol that provides reliable and ordered delivery of data between devices.

UDP - User Datagram Protocol

UDP is a connectionless transport protocol that sends data without establishing a connection. It is faster but does not guarantee delivery.

Nmap:

Nmap (Network Mapper) is a network scanner tool used to discover live hosts, identify open ports, gather information about network services.

Procedure:

1. Open the assigned TryHackMe room and start the target machine.
2. Open the terminal provided in the TryHackMe environment.
3. Note the target IP address provided by TryHackMe.
4. Perform ARP, ICMP, TCP and UDP host discovery scan using Nmap.
5. Observe the output and identify the hosts as shown in "Host is up".

Nmap Commands used

1. General live Host Discovery

Command: nmap -su 10.49.151.126

Observation: The target host was detected as active

2 ARP scan:

command: nmap -sn -PR 10.49.151.126

observation: The ARP scan successfully detected the target as a live host.

3 ICMP scan

command: nmap -sn -PE 10.49.151.126

observation: The ICMP scan detected the target as a live host.

4. TCP SYN ping

command: nmap -sn -PS 10.49.151.126

observation: The TCP SYN scan detected the target as a live host.

5 Tcp ACK ping

command: nmap -sn -PA 10.49.151.126

observation: The TCP ACK scan detected the target as a live host

6. UDP ping

command: nmap -sn -PU 10.49.151.126

observation: The UDP scan detected the target as a live host.

RESULT:

The discovery of live/active hosts in a network using Nmap by performing ARP scan, Icmp scan, TCP/UDP ping scan in TryHackme platform was verified successfully

DATE: 11/09/2026

Ex No. 11

Examine Network Address Translation (NAT) using Cisco Packet Tracer

Aim:

Implement Network Address Translation (NAT) using Cisco Packet Tracer. We'll configure static, dynamic, and Port Address Translation (PAT) on a router to enable communication between a private internal network and a public external network.

Procedure:

Step 1: Setup the Network Topology

Step 2: Add Devices

Step 3: Connect Devices

Step 4: Verify Connections

Implementation of NAT using Cisco Packet Tracer

1. Open Cisco Packet Tracer and place two PCs, a switch, a router, and an outside network device. Connect them using suitable cables.

2. Assign IP addresses.

a) PC1: 192.168.1.10/24

b) PC2: 192.168.1.11/24

c) Router inside Interface: 192.168.1.1/24

d) Router outside Interface: 200.1.1.1/24

e) Outside device: 200.1.1.2/24

3. Configure the router interfaces using CLI commands and enable them using no shutdown.

4. Configure NAT by identifying the inside and outside interfaces:

```
interface GigabitEthernet 0/0
ip nat inside
exit
interface GigabitEthernet 0/1
ip nat outside
exit
```

5. Configure static NAT to map a private IP to a public IP:

```
ip nat inside source static
192.168.1.10 200.1.1.10
```

6. Configure Dynamic NAT by creating a public IP pool and access list.

```
ip nat pool MY_POOL 200.1.1.20
200.1.1.25 netmask 255.255.255.0
access-list 1 permit 192.168.1.0
0.0.0.255
ip nat inside source list 1 pool
MY_POOL
```

7. Configure PAT to allow multiple ~~de~~ private ~~devices~~ hosts to share one public IP:

```
ip nat inside source list 1
interface GigabitEthernet 0/1 overload
```

8. Set the default gateway of both PCs to 192.168.1.1.

9. Verify NAT operation using show ip nat translations and test connectivity by pinging the outside device. Use Simulation Mode to observe packet translation.

Output:

1. Network Topology
2. NAT Translation Table
3. Connectivity Test
4. PAT configuration Output

RESULT.

NAT was configured in Cisco Packet Tracer using Static NAT, Dynamic NAT, and PAT, enabling private network hosts to communicate with the external network.

Ex No: 13

Date: 28/8/2026

DEMONSTRATE NETWORK FORENSICS USING PCAPXRAY TOOL

AIM:

To analyze and visualize network traffic from a PCAP file using the PCAPXray tool for forensic investigation.

STEPS TO COMPLETE NETWORK FORENSICS EXPERIMENT USING PCAPXRAY

Step 1: Prepare Your Computer

i) Install Python 3

Step 2: Install PcapXray Tool

i) Open cmd or Terminal (Mac/Linux)

ii) Type this command and press Enter

iii) pip install pcapxray

This installs the Pcapxray tool on your computer.

Step 3: Get a PCAP File (Network Traffic Capture)

Option A: use a sample PCAP file

Option B: Capture your own traffic (optional)

Step 4: Run PcapXray on your PCAP File

i) Put the PCAP file in a folder you can easily access.

ii) Open cmd/Terminal and navigate to that folder.

Example:

cd path_to_your_folder

Run this command

pcapxray -f capture.pcap

Replace capture.pcap with the name of your PCAP file

Step 5: View the Results

i) After the command runs, it will create a file named pcapxray.html.

ii) Open this file in your web browser

iii) You will see a visual map of network traffic showing

* computers (hosts)
* connections between them
* Protocols and ports used

Step 6: Analyze and See

i) Look for the biggest nodes - these are the most active devices.

ii) Check if any strange IP addresses are communicating. Notice unusual ports or repeated connections.

Result:

Pcapxray successfully generated a visual map of hosts, connections, and protocols, helping identify suspicious or abnormal network activities.

Ex No: 14

Date: 11/10/2026

To CAPTURE, SAVE, AND ANALYSE NETWORK TRAFFIC ON TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS USING WIRESHARK TOOL

AIM:

The aim of this experiment is to capture, save, and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS using Wireshark Tool.

PROCEDURE:

1. Download and install Wireshark from its official website. Ensure you have administrator privileges for network capture.

2. Launch Wireshark and identify the network interface (such as Ethernet or Wi-Fi) through which your device is connected to the network.

3. Before starting the capture, apply capture filters to focus on specific protocols and reduce unwanted data.

Ex:

Capture only TCP traffic : tcp

Capture only UDP traffic : udp

Capture only HTTP traffic : tcp port 80

Capture only DNS traffic : udp port 53

capture only ICMP traffic : icmp

capture only ARP traffic : arp

capture DHCP traffic : port 67 or port 68

4. click the start button to begin capturing live network traffic on the chosen interface. Wireshark will start displaying packets in real time.

5. While capturing, use display filters to isolate and analyze specific protocols more easily.

Ex:

To display only TCP packets : tcp

To display only UDP packets : udp

To display only HTTP packets : http

To display only DNS packets : dns

To display only ICMP packets : icmp

To display only ARP packets : arp

To display only DHCP packets : bootp

6. Let the capture run until you have collected enough packets for analysis, then click the Stop button to halt the capture process.

7. Save the captured traffic by navigating to File > Save As, and choose a suitable filename and format (usually .pcap or .pcapng).

8. Review and analyze the captured packets in detail. Look for key information such as source and destination IP addresses, protocol types, packet lengths and flags. This analysis can help identify normal traffic flows or detect anomalies and potential issues within the network.

OUTPUT:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.1	8.8.8.8	ICMP	56	Echo Requ
2	0.002345	8.8.8.8	192.168.1.1	ICMP	64	Echo Reply
3	0.003234	192.168.1.2	192.168.1.1	ARP	42	Who has 192.168.1.

RESULT:

Thus, this experiment capture, save, and analyze network traffic on TCP / UDP / IP / HTTP / ARP / DHCP / ICMP / DNS using Wireshark tool.

Ex No: 15 :

DATE. 8/10/2026

ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL

AIM

To analyze the usage statistics of different types of servers (Apache, Nginx, FTP/Proxy) using the Webalizer tool.

PROCEDURE:

1. Install the Webalizer tool in the system.
2. Verify installation
3. Locate the log files of the servers
4. Run Webalizer on the log file to generate a report
5. Open a web browser and access the report
6. Repeat the same process for Nginx, FTP, or Proxy logs.
7. Compare and analyze the report (hits, visits, bandwidth, top URLs, and countries).

PROGRAM:

1. Install Webalizer

   sudo apt-get install webalizer

2. Run Webalizer for Apache logs

   sudo webalizer /var/log/apache2/access.log -o /var/www/html/apache_report/

3. Run Webalizer for Nginx logs

   sudo webalizer /var/log/nginx/access.log -o/var/www/html/nginx_report/

4. Run Webalizer for Proxy logs sudo webalizer /var/log/squid/access.log -o/var/www/html/proxy_report/.

Output

Apache	12,560, 1240	1.8 GiB	192.168.1.10	http://localhost/apache_report/
Nginx	98409 80	1.2 GiB	192.168.1.26	http://localhost/nginx_report/
Proxy	15,300 1450	2.1 GiB	192.168.1.30	http://localhost/proxy_report/

Graphical Output

Monthly summary chart

Daily/hourly usage histogram

Pie chart of top hosts and URLs

RESULT:

Thus, the Webalizer tool successfully generated graphical and statistical reports of different servers. The analysis provided insights into: Total hits and visits, Bandwidth usage, TOP IPs and URLS, Daily, monthly and hourly traffic.